Abubakar_Mubarak_301927457_FinalDocumentation

This portfolio designs, builds, and distributes the database for Muhab Gadget Store, using PostgreSQL 16 and MongoDB 8.0 in GitHub Codespaces.

Abubakar Mubarak (Student ID 301927457) · Master of Information Technology, MIVA Open University · MIT 8103 Advanced Database Systems, 2026/2027 First Semester

Case study: Muhab Gadget Store · **GitHub:** github.com/Mouthytech/Abubakar_Mubarak_301927457

1. Selected case study: Muhab Gadget Store

Muhab is a simulated online store selling smartphones, laptops, tablets, audio devices, wearables and accessories. Customers browse the catalogue, order online, pay by card, bank transfer or on delivery, and receive items through delivery partners.

Its users are customers, administrators, inventory staff, customer support and delivery partners. It stores customers, products and their specifications, stock, orders, payments, deliveries and reviews, and supports placing orders (stock checked and reduced in one transaction), payments and refunds, delivery tracking and sales reporting.

2. Portfolio activities

All five activities use the Muhab case study; full scripts, outputs and screenshots are in each repository folder.

Portfolio 1: Database Design and Modelling. I designed 10 tables with an ER diagram, normalised them to 3NF/BCNF, and enforced primary, foreign, unique and CHECK constraints. All 9 deliberately invalid operations were rejected.

Portfolio 2: Query Processing and Optimisation. With 50,010 orders loaded, I ran eight business queries and compared EXPLAIN ANALYZE plans before and after four indexes.

Test	Index	Before	After
Customer order history	orders(customer_id)	2.228 ms	0.271 ms
Oldest pending orders	Partial index on pending orders (56 kB)	3.239 ms	0.081 ms

Test	Index	Before	After
Sales of one product	Covering index (Index Only Scan)	5.650 ms	2.242 ms
Weekly sales report	orders(order_date)	14.538 ms	18.791 ms
Revenue by category	None useful	70.574 ms	71.039 ms

The weekly report did not improve: its bottleneck was the join, not the date filter.

Portfolio 3: Transactions and Concurrency. I demonstrated COMMIT, ROLLBACK, automatic rollback on a failed stock check, and a SAVEPOINT. Two sessions buying the last phone caused a lost update; SELECT ... FOR UPDATE and REPEATABLE READ each prevented it.

Portfolio 4: NoSQL and Advanced Data Models. I modelled the varied product catalogue as 16 MongoDB documents with a $jsonSchema validator, and carts as key-value documents with a 7-day TTL index. A compound index cut documents examined from 16 to 3. Orders and stock stay in PostgreSQL, which handles transactions and locking.

Portfolio 5: Distributed and Cloud Database. A 3-node MongoDB replica set replicated majority writes to every node. When the primary was shut down, a new one was elected, and the old primary rejoined with 0 seconds of lag. A design document covers CAP, sharding and cloud deployment.

3. Individual technical reflection

Design choices. I kept product specifications out of PostgreSQL because a phone, a laptop and a power bank share almost no attributes. I stored the price paid in order_items instead of an order total, left foreign keys unindexed so I could measure indexing in Portfolio 2, and kept stock only in PostgreSQL so it has one source of truth.

Challenges and how I solved them.

- **MongoDB error "The $round accumulator is a unary operator":** I moved the rounding out of $group into a $set stage and committed the fix. The error message said exactly which stage was wrong.
- **The date index made the weekly report no faster:** reading the whole plan showed the slow step was the join over 100,015 order items. An index only helps if it removes the bottleneck.

- **Session B waited for a lock, but the phone was still sold twice:** comparing it with FOR UPDATE and REPEATABLE READ showed that a row lock protects the row, not a decision made from an older read.
- **A new primary was elected in 0 seconds:** a clean shutdown hands over to a secondary; a crash would wait about 10 seconds.

Lessons learned. Measuring changed my view of indexes: the smallest index (56 kB, partial) gave the biggest gain, and two indexes gave none. I also learned that the right database depends on the data. Orders need transactions and constraints, while a varied product catalogue fits documents better. Finally, committing after each step gave me a clear record of my progress, and it made mistakes easy to trace and fix.

4. AI use declaration

- **Tool used:** Claude (Anthropic), through the claude.ai chat interface.
- **What it was used for:** a step-by-step guide through all five activities. It suggested the table design, wrote drafts of the SQL, MongoDB and shell scripts, explained errors, and drafted the README files, analysis documents and this summary, including a first draft of the reflection based on the problems I reported.
- **How I reviewed and verified the output:** I ran every script in my own Codespace and checked each result against the expected output before committing. When something failed, I reported the actual error and re-ran the fix (for example, the MongoDB $round error). All figures come from my own runs. I took the screenshots, reviewed and edited the documentation and reflection, and can explain each decision and result.

5. References

1. MIVA Open University (2026) *MIT 8103 Advanced Database Systems: First Semester Continuous Assessment brief*.
2. Harrison, P. (2026) *MIT 8103 Advanced Database Systems Codelab*. GitHub. github.com/arizonal2014/MIT8103_Codelab (step-by-step method followed).
3. PostgreSQL Global Development Group (2026) *PostgreSQL 16 Documentation*. postgresql.org/docs/16.
4. MongoDB, Inc. (2026) *MongoDB Manual (version 8.0)*. mongodb.com/docs/manual.
5. Federal Republic of Nigeria (2023) *Nigeria Data Protection Act 2023*.